Unit Testing of Calculator Class Using MSTest

Name: Diksha Chaudhary

Environment: .NET CLI with Visual Studio Code on macOS

Course: *(*Advanced C# and Introduction to .Net*)*

Date: *(*22/02/2026*)*

---

1. Objective

The objective of this assignment is to implement a calculator using a class library and verify its functionality through unit testing. Unit testing is used to ensure that each arithmetic operation returns the correct result and that exceptional scenarios such as division by zero are handled properly.

---

2. Tools and Technologies Used

- C#
- .NET SDK
- MSTest Framework
- Visual Studio Code (macOS)

---

3. Project Structure

The solution consists of two separate projects:

- **CalculatorLibrary** – Contains the implementation of the calculator methods.
- **CalculatorLibrary.Tests** – Contains unit test methods for validating the functionality.

This separation helps in maintaining clean code and makes the application easier to test and manage.

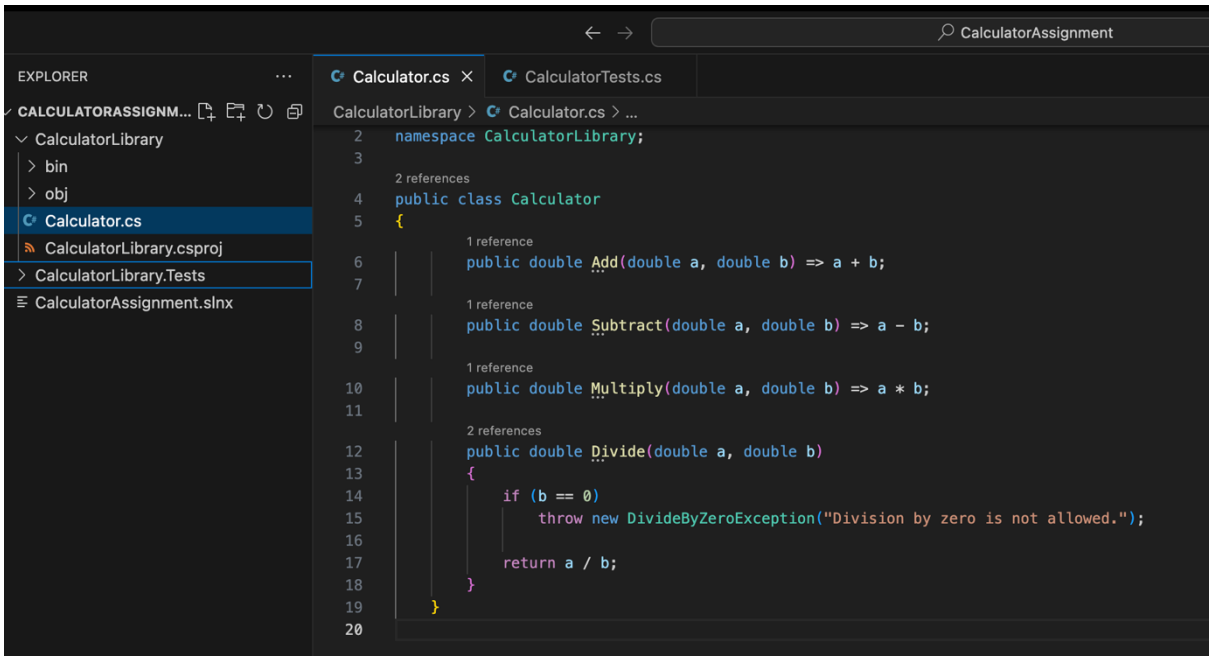
---

4. Implementation

The calculator class includes the following methods:

- Addition
- Subtraction
- Multiplication
- Division

An exception is thrown when division by zero is attempted. This prevents invalid operations and ensures that the program behaves correctly during runtime.



---

5. Unit Testing

Unit testing is implemented using the MSTest framework.

The test cases verify:

- Correct results for valid inputs
- Operations involving zero
- Handling of division by zero

Each test method compares the expected result with the actual output returned by the calculator.

CalculatorAssignment

EXPLORER

CALCULATORASSIGNM...

- CalculatorLibrary
- CalculatorLibrary.Tests
  - bin
  - obj
  - TestResults
  - CalculatorLibrary.Tests.csproj
  - CalculatorTests.cs
  - MSTestSettings.cs
- CalculatorAssignment.slnx

Calculator.cs | CalculatorTests.cs

CalculatorLibrary.Tests > CalculatorTests.cs > CalculatorTests > Add_ReturnsCorrectResult

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using CalculatorLibrary;
using System;

namespace CalculatorLibrary.Tests
{
    [TestClass]
    0 references
    public class CalculatorTests
    {
        6 references
        private Calculator _calculator = null!;

        [TestInitialize]
        0 references
        public void Setup()
        {
            _calculator = new Calculator();
        }

        [TestMethod]
        0 references
        public void Add_ReturnsCorrectResult()
        {
            Assert.AreEqual(8, _calculator.Add(5, 3));
        }

        [TestMethod]
        0 references
        public void Subtract_ReturnsCorrectResult()
        {
            Assert.AreEqual(6, _calculator.Subtract(10, 4));
        }

        [TestMethod]
        0 references
        public void Multiply_ReturnsCorrectResult()
        {
            Assert.AreEqual(30, _calculator.Multiply(6, 5));
        }

        [TestMethod]
        0 references
        public void Divide_ReturnsCorrectResult()
        {
            Assert.AreEqual(5, _calculator.Divide(20, 4));
        }
```

CalculatorAssignment

EXPLORER

CALCULATORASSIGNM...

- CalculatorLibrary
- CalculatorLibrary.Tests
  - bin
  - obj
  - TestResults
  - CalculatorLibrary.Tests.csproj
  - CalculatorTests.cs
  - MSTestSettings.cs
- CalculatorAssignment.slnx

Calculator.cs | CalculatorTests.cs

CalculatorLibrary.Tests > CalculatorTests.cs > CalculatorTests > Add_ReturnsCorrectResult

```
    public class CalculatorTests

        [TestMethod]
        0 references
        public void Divide_ByZero_ThrowsException()
        {
            try
            {
                _calculator.Divide(10, 0);
                Assert.Fail("Expected DivideByZeroException was not thrown.");
            }
            catch (Exception ex)
            {
                Assert.IsInstanceOfType(ex, typeof(DivideByZeroException));
            }
        }
    }
}
```

6. Test Execution

All test cases were executed using the following command in the terminal: **dotnet test**

The output shows that all tests passed successfully.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    TEST RESULTS    PORTS                                                    zsh
(base) dikshachaudhary@Dikshas-MacBook-Air CalculatorAssignment % dotnet build
Restore complete (0.8s)
  CalculatorLibrary net10.0 succeeded (0.1s) → CalculatorLibrary/bin/Debug/net10.0/CalculatorLibrary.dll
  CalculatorLibrary.Tests net10.0 succeeded (2.6s) → CalculatorLibrary.Tests/bin/Debug/net10.0/CalculatorLibrary.Tests.dll

Build succeeded in 3.6s
(base) dikshachaudhary@Dikshas-MacBook-Air CalculatorAssignment % dotnet test
Restore complete (0.5s)
  CalculatorLibrary net10.0 succeeded (0.1s) → CalculatorLibrary/bin/Debug/net10.0/CalculatorLibrary.dll
  CalculatorLibrary.Tests net10.0 succeeded (0.2s) → CalculatorLibrary.Tests/bin/Debug/net10.0/CalculatorLibrary.Tests.dll
Test Parallelization enabled for /Users/dikshachaudhary/CalculatorAssignment/CalculatorLibrary.Tests/bin/Debug/net10.0/CalculatorLibrary.Tests.dll (Workers: 8, Scope: MethodLevel)
  CalculatorLibrary.Tests test net10.0 succeeded (0.8s)

Test summary: total: 5, failed: 0, succeeded: 5, skipped: 0, duration: 0.7s
Build succeeded in 1.7s
(base) dikshachaudhary@Dikshas-MacBook-Air CalculatorAssignment %
```

---

7. Result

All five unit test cases executed successfully.

The calculator performs all arithmetic operations correctly and handles division by zero as expected.

---

8. ConclusionS

Through this assignment, I learned how to create a reusable class library and test it using a unit testing framework. Unit testing improves the reliability of the application and helps detect errors at an early stage. It also ensures that future changes do not break the existing functionality.